

Universidad Galileo	Guatemala 3 de mayo 2026
Facultad: FISICC	Alumno: Luis Cordova
Curso: Sistemas de Arquitectura	Carnet: 23012122
Sección: A	Hora de Laboratorio: 1:00 a 1:50
Auxiliar: Brandon Eduardo Mendoza Espinoza	Día de Laboratorio: Lunes

Laboratorio #1 Getting Started with STM32 Nucleo

Resumen:

En esta práctica se aprendió como configurar, desarrollar y depurar una aplicación en lenguaje C para microcontroladores de núcleo ARM Cortex-M4, utilizando Visual Studio Code junto con el entorno Keil MDK v6. Se creó una solución CMSIS seleccionando el procesador genérico ARM Cortex-M4 y la plantilla "Blank solution", verificando que estuvieran seleccionados los componentes de software CMSIS CORE y Device Startup. Posteriormente se practicó el uso de operadores, punteros, funciones y archivos de cabecera en C mediante la creación de un header propio llamado "utilities.h", que contiene las funciones bitSet, bitClear, bitToggle y stringLength. Dentro de la rutina principal se escribió código que demuestra el correcto funcionamiento de cada una de estas funciones, mostrando los resultados con la función printf y terminando todos los strings con los caracteres de retorno de carro y salto de línea. Finalmente se configuró el entorno de Arm Tools y una configuración de depuración con Model Debug Connection para simular la ejecución del código, verificando la salida del programa en la consola de depuración de VS Code.

Objetivos:

- Aprender el procedimiento para configurar, desarrollar y depurar una aplicación en lenguaje C para microcontroladores de núcleo ARM Cortex-M utilizando Visual Studio Code con el entorno Keil MDK v6.
- Implementar un archivo de cabecera propio en C que agrupe funciones reutilizables, aplicando correctamente las guardas de inclusión.
- Verificar el funcionamiento de la aplicación mediante simulación en el modelo Arm Fixed Virtual Platform, empleando la función printf para observar los resultados en la consola de depuración.

Teoría:

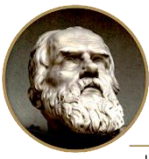
Visual Studio Code junto con Keil MDK v6 proporciona un entorno de desarrollo para procesadores ARM Cortex-M, permitiendo editar, compilar y depurar aplicaciones mediante el Arm Keil Studio Pack [1].

El procesador Cortex-M4, basado en la arquitectura Armv7-M, utiliza instrucciones Thumb-2 y registros de 32 bits, lo que facilita la manipulación de registros y operaciones a nivel de bit, características ampliamente utilizadas en sistemas embebidos [2].

Los proyectos se organizan mediante los archivos `csolution.yml` y `cproject.yml`, que definen la configuración del proyecto, los componentes de software y los archivos fuente. Entre los componentes principales se encuentran CMSIS CORE y Device Startup, necesarios para la inicialización y el funcionamiento del sistema [3].

El desarrollo en C hace uso de punteros, operadores a nivel de bit y archivos de cabecera, herramientas fundamentales para acceder a memoria, manipular registros y organizar el código. Asimismo, los tipos `uint32_t` y `uint8_t` definidos en `stdint.h` garantizan un tamaño fijo de los datos, requisito importante en aplicaciones embebidas [4], [5].

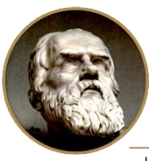
La ejecución y depuración del programa pueden realizarse mediante un Fixed Virtual Platform (FVP), el cual simula el comportamiento del procesador sin necesidad de utilizar hardware físico [6].



A continuación, se presentan las capturas obtenidas durante el desarrollo y la ejecución de la práctica.

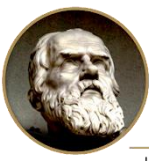
```
Lab01_LuisCordova_23012122 > C utilities.h
1  #ifndef UTILITIES_H
2  #define UTILITIES_H
3
4  #include <stdint.h>
5
6  void bitSet(uint32_t *ptr, uint8_t bit);
7
8  void bitClear(uint32_t *ptr, uint8_t bit);
9
10 void bitToggle(uint32_t *ptr, uint8_t bit);
11
12 uint8_t stringLength(uint8_t *str);
13
14 #endif
```

Ilustración 1- se declaran las funciones utilizadas en el programa.



```
Lab01_LuisCordova_23012122 > C utilities.c
1  #include "utilities.h"
2
3  void bitSet(uint32_t *ptr, uint8_t bit) {
4      *ptr |= (1UL << bit);
5  }
6
7  void bitClear(uint32_t *ptr, uint8_t bit) {
8      *ptr &= ~(1UL << bit);
9  }
10
11 void bitToggle(uint32_t *ptr, uint8_t bit) {
12     *ptr ^= (1UL << bit);
13 }
14
15 uint8_t stringLength(uint8_t *str) {
16     uint8_t count = 0;
17     while (str[count] != '\0') {
18         count++;
19     }
20     return count;
21 }
```

Ilustración 2-Implementación de funciones en utilities.c



```
uisCordova_23012122 > C main.c
#include "RTE_Components.h"
#include CMSIS_device_header
#include <stdio.h>
#include "utilities.h"

int main() {

    uint32_t reg = 0;
    uint8_t texto[] = "Hola mundo";

    printf("Inicial: %d\r\n", reg);

    bitSet(&reg, 3);
    printf("bitSet(3): %d\r\n", reg);

    bitClear(&reg, 3);
    printf("bitClear(3): %d\r\n", reg);

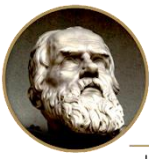
    bitToggle(&reg, 4);
    printf("bitToggle(4): %d\r\n", reg);

    bitToggle(&reg, 4);
    printf("bitToggle(4): %d\r\n", reg);

    printf("Largo de \"%s\": %u\r\n", texto, strlen(texto));

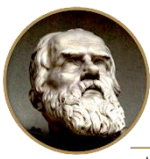
    for (;;) {
    }
}
```

Ilustración 3- Implementación del programa principal en main.c



```
Connected to stopped target Cortex-M4
>cd "C:\micro labs\micro\Blank_solution"
Working directory "C:\micro labs\micro\Blank_solution"
Semihosting server socket created at port 8000
Semihosting enabled automatically due to semihosting symbol detected in i
Waiting for execution to stop at: main
Execution stopped in Privileged Thread mode at breakpoint 1: 0x000020E8
In main.c
0x000020E8  6,0  int main() {
Deleted temporary breakpoint: 1
>core 1
Current core is Cortex-M4 (ID 1) - stopped in main() at main.c:6
[model] Inicial: 0
[model] bitSet(3): 8
[model] bitClear(3): 0
[model] bitToggle(4): 16
[model] bitToggle(4): 0
[model] Largo de "Hola mundo": 10
```

Ilustración 4 - Resultado de la ejecución del programa



Descripción del código implementado:

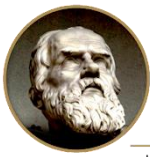
El enunciado de la práctica solicita la creación de un archivo de cabecera utilities.h con cuatro funciones específicas, cuyas especificaciones se resumen a continuación.

- `void bitSet(uint32_t *ptr, uint8_t bit)`: recibe la dirección de memoria de una variable entera sin signo de 32 bits y un número entre 0 y 31 que indica el bit a modificar, reemplazando el valor anterior de ese bit por un 1.
- `void bitClear(uint32_t *ptr, uint8_t bit)`: recibe los mismos argumentos y reemplaza el valor anterior del bit indicado por un 0.
- `void bitToggle(uint32_t *ptr, uint8_t bit)`: recibe los mismos argumentos y reemplaza el valor anterior del bit indicado por su valor negado; es decir, si el bit vale 0 se sustituye por 1 y viceversa.
- `uint8_t stringLength(uint8_t *str)`: recibe un string como argumento y retorna la cantidad de caracteres que forman parte de este.

A continuación, se describe cada uno de los archivos que conforman la aplicación desarrollada.

Archivo utilities.h – Ilustracion 1

Este archivo de cabecera declara los prototipos de las cuatro funciones de utilería implementadas en la práctica. Las guardas de inclusión formadas por las directivas `#ifndef UTILITIES_H`, `#define UTILITIES_H` y `#endif` impiden que el contenido se procese más de una vez cuando la cabecera es incluida desde varios archivos fuente, lo que evitaría errores de redefinición. Se incluye la cabecera estándar `stdint.h` para disponer de los tipos de ancho fijo `uint32_t` y `uint8_t`, los cuales garantizan que las variables tengan exactamente 32 y 8 bits respectivamente en cualquier plataforma.



Funciones de manipulación de bits (utilities.c) - Ilustracion 2

Las tres funciones reciben como argumentos un puntero a una variable entera sin signo de 32 bits y un número entre 0 y 31 que indica la posición del bit a modificar. Al trabajar con el puntero y el operador de indirección, la modificación se realiza directamente sobre la variable original declarada en la función principal. En las tres funciones la expresión 1UL desplazada a la izquierda construye una máscara con un único bit en 1 en la posición solicitada. La función bitSet aplica el operador OR para forzar ese bit a 1; bitClear aplica el operador AND junto con la máscara negada para forzarlo a 0; y bitToggle aplica el operador XOR para invertir el valor que tuviera previamente. En los tres casos el resto de los bits de la variable permanece sin alteración, lo cual constituye la razón principal para utilizar operaciones a nivel de bit en lugar de asignaciones directas.

Función strlen (utilities.c) - Ilustracion 2

Esta función recibe un puntero al primer carácter de una cadena y recorre las posiciones consecutivas de memoria incrementando un contador, hasta encontrar el carácter nulo terminador que en C marca el final de toda cadena. El valor devuelto corresponde al número de caracteres útiles, sin incluir dicho terminador, replicando así el comportamiento de la función strlen de la biblioteca estándar. El recorrido es posible porque los caracteres de una cadena se almacenan en posiciones contiguas de memoria.

Rutina principal (main.c) — Ilustración 3 y resultados de ejecución — Ilustración 4

La función principal demuestra el funcionamiento de cada una de las funciones desarrolladas. Se declara la variable reg inicializada en cero y una cadena de prueba, y a cada función se le pasa la dirección de reg mediante el operador &, lo cual permite que las modificaciones se reflejen en la variable original. La secuencia de pruebas activa el bit 3, con lo que la variable pasa de 0 a 8; a continuación, lo limpia, regresando a 0; luego invierte el bit 4 obteniendo el valor 16, y al invertirlo por segunda vez retorna a 0, evidenciando el carácter reversible de la operación XOR. Finalmente se imprime la longitud de la cadena de prueba. Cada resultado se muestra mediante printf y todos los mensajes terminan con la secuencia de retorno de carro y salto de línea, tal como lo solicita el enunciado de la práctica. El programa concluye con un bucle infinito, ya que en un sistema embebido la función main no debe retornar: el procesador debe permanecer siempre ejecutando instrucciones.

Conclusiones:

- Aprendí que configurar correctamente el entorno de desarrollo es una parte del trabajo tan importante como escribir el código. Seguir el procedimiento de crear la solución CMSIS, verificar los componentes CMSIS CORE y Device Startup, y configurar el entorno de Arm Tools con el Arm Debugger y el Arm Virtual Hardware me permitió entender que cada una de esas piezas cumple una función específica dentro del flujo de compilación y depuración, y no son simples pasos que se repiten de memoria.
- Comprendí la utilidad real de los punteros al ver que las funciones bitSet, bitClear y bitToggle modifican la variable original declarada en main. Si las funciones hubieran recibido el valor en lugar de la dirección, habrían trabajado sobre una copia y el cambio se habría perdido al retornar. Observar en la consola cómo la variable pasaba de 0 a 8, regresaba a 0 y luego a 16 fue lo que me hizo entender de forma concreta la diferencia entre pasar por valor y pasar por referencia.
- Aprendí a organizar el código separando la declaración de la implementación mediante un archivo de cabecera propio. Además, comprendí la función de las guardas de inclusión, que en un inicio me parecían una formalidad innecesaria, pero que evitan errores de redefinición cuando la misma cabecera se incluye desde varios archivos.

Bibliografía:

- [1] Arm Ltd., "Arm Keil Studio for VS Code — User's Guide," 2026. [En línea]. Disponible: <https://mdk-packs.github.io/vscode-cmsis-solution-docs/> 8
- [2] Arm Ltd., Arm Cortex-M4 Processor Technical Reference Manual, Revision r0p1, 2020. [En línea]. Disponible: <https://developer.arm.com/documentation/100166/0001/>
- [3] Open-CMSIS-Pack, "CMSIS-Toolbox — CSolution Project Format," 2026. [En línea]. Disponible: <https://open-cmsis-pack.github.io/cmsis-toolbox/YML-Input-Format/>
- [4] B. W. Kernighan y D. M. Ritchie, The C Programming Language, 2.^a ed. Englewood Cliffs, NJ: Prentice Hall, 1988.
- [5] ISO/IEC 9899:2018, Information technology — Programming languages — C, International Organization for Standardization, Ginebra, Suiza, 2018.
- [6] Arm Ltd., "Arm FVP models: Overview," Arm Virtual Hardware Documentation, 2026. [En línea]. Disponible: <https://arm-software.github.io/AVH/main/simulation/html/index.html> [7] Arm Ltd., "Arm Keil MDK v6 Editions," 2026. [En línea]. Disponible: